



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 04

Objective & Lecture Mapping: In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment [cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

Part 1: Practical Implementation — Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

Informed search algorithms require a heuristic function, $h(n)$, which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called `Week_04` in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class [cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula $h(n) = |x_1 - x_2| + |y_1 - y_2|$ and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

Step 1.2: Implementing A* Search

A* evaluates nodes by combining the path cost so far, $g(n)$, and the estimated cost to the goal, $h(n)$. The evaluation function is $f(n) = g(n) + h(n)$.

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses $g(n)$, your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, $g(n) = 0$. Calculate $h(n)$ using your chosen heuristic method, and set $f(n) = g(n) + h(n)$.
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos` to `reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate $g_{\text{new}} = g_{\text{current}} + 1$, h_{new} using your heuristic, and $f_{\text{new}} = g_{\text{new}} + h_{\text{new}}$. Push the new tuple to the priority queue.

Step 1.3: Integrating A* into the Agent's Decision Loop

Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent`.
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos`. Call your `astar_search` method and store the returned list of actions in `self.plan`.

5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()`. Run the simulation and observe the optimized pathfinding!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

UCS prioritises nodes using only $g(n)$, the cost already spent to reach a node from the start, so it expands outward from the start in all directions without any awareness of where the goal lies. A* instead prioritises by $f(n) = g(n) + h(n)$, combining the cost so far with a heuristic estimate $h(n)$ of the remaining cost to the goal, so it favours nodes that are both cheap to reach and appear close to the goal. This makes UCS an uninformed search and A* an informed one: both return an optimal solution (provided h is admissible, and consistent for graph search), but A* typically expands far fewer nodes because the heuristic steers it toward the goal. In fact, UCS is just A* with $h(n) = 0$ — the least informed case of the same evaluation function.

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

Why Manhattan Distance is admissible here

A heuristic is admissible if it never overestimates the true remaining cost, i.e. $h(n) \leq h^*(n)$ for every node n . On this grid the agent can only move up, down, left or right, and every move costs 1 while changing exactly one coordinate by exactly one unit. To get from a node $n = (x_1, y_1)$ to the goal (x_2, y_2) , the agent must therefore make at least $|x_1 - x_2|$ horizontal moves and at least $|y_1 - y_2|$ vertical moves — that is exactly the Manhattan

Distance. No shorter route exists even on a completely empty grid, and walls or obstacles can only force detours that make the real path longer, never shorter. So Manhattan Distance is a lower bound on the true cost: $h(n) \leq h^*(n)$ always holds, making it admissible. It is also consistent, since moving to any neighbour changes h by at most 1 while incurring a step cost of 1, satisfying $h(n) \leq c(n, a, n') + h(n')$. This is why A* with this heuristic is guaranteed optimal even under graph search. (Note that Manhattan Distance would *not* be admissible if diagonal movement were allowed, because it would then overestimate the cost of diagonal shortcuts.)

What happens if the heuristic is not admissible

If $h(n)$ overestimates the true cost to the goal for some node, A* loses its optimality guarantee. The proof of optimality relies on the fact that any node n on the optimal path satisfies $f(n) \leq \text{cost of the optimal solution}$, so a suboptimal goal G_2 can never be selected ahead of it. With an overestimating heuristic, the f -value of a node on the optimal path can be inflated above the f -value of a suboptimal goal, so A* pops the suboptimal goal first and returns a more expensive path than necessary. The algorithm would still terminate and still find a valid path — it may even expand fewer nodes and run faster, since aggressive overestimation pushes the search to behave more like Greedy Best-First Search — but the path it returns is no longer guaranteed to be the shortest one. In short, inadmissibility trades correctness for speed.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

No, Manhattan Distance would no longer be admissible under 8-way movement, because a diagonal move reduces both the x and y difference simultaneously while Manhattan Distance counts them as two separate steps — for example, travelling from $(0, 0)$ to $(3, 3)$ has a Manhattan Distance of 6 but a true cost of only 3 diagonal moves, so the heuristic overestimates $h^*(n)$ and A* would lose its guarantee of returning the shortest path. The correct metric is Chebyshev Distance, $h(n) = \max(|x_1 - x_2|, |y_1 - y_2|)$, since the agent needs at least as many moves as the larger coordinate difference; this equals the true cost on an open grid and

underestimates whenever obstacles force a detour, making it both admissible and consistent, so A^* stays optimal. (If diagonal moves are charged $\sqrt{2}$ rather than 1, the octile distance $h(n) = (dx + dy) + (\sqrt{2} - 2) \cdot \min(dx, dy)$ is the better choice, as Chebyshev would then underestimate more than necessary and expand extra nodes.)

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

A stronger heuristic is to take the **maximum over all remaining food items of (distance from the agent to that item)**, or better, to build a **minimum-spanning-tree (MST) bound**: compute the Manhattan distance from the agent's current position to the nearest remaining food, then add the cost of a minimum spanning tree connecting all remaining food items to each other, giving $h(n) = \text{dist}(\text{agent}, \text{nearest food}) + \text{MST}(\text{remaining food})$. This is admissible because any complete tour that eats all the food must, at minimum, travel from the agent to some food item and then connect every remaining item into one path — and since a path visiting all food items is itself a spanning tree of those nodes, the cheapest spanning tree can never cost more than the true optimal route. It dominates the closest-food heuristic ($h_{\text{MST}} \geq h_{\text{closest}} \geq 0$ while still never overestimating), so it expands strictly fewer nodes, because it accounts for the total remaining work rather than only the first meal; a simpler but weaker alternative is the maximum single-food distance, which is cheaper to compute but ignores the cost of collecting everything else. The trade-off is computation time — the MST must be recomputed as food is consumed — but the reduction in expanded nodes generally outweighs the per-node cost.

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

Finalize and Lock Lab 04 Documentation



FACULTY OF COMPUTING